EcoControl

Cidades ESG Inteligentes - DevOps e CI/CD

Victor Hugo Bastos Colomera - RM 566171

Documentação da etapa de containerização, testes e automação de deploy

O objetivo desta entrega é aplicar práticas de DevOps ao projeto EcoControl sem alterar a proposta funcional do sistema. A aplicação foi organizada para executar em containers, rodar testes automaticamente e possuir fluxos separados para staging e produção.

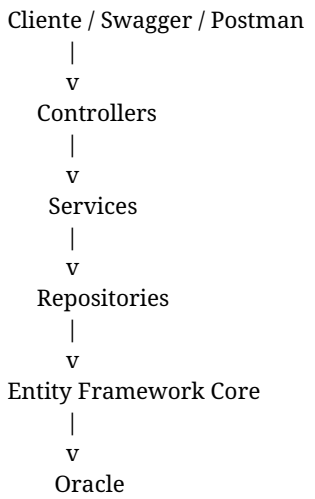
1. Contexto do projeto

O EcoControl é uma API voltada ao acompanhamento do consumo de energia de equipamentos industriais. O sistema registra os equipamentos, horários de operação, leituras de sensores, desligamentos e informações de consumo e economia de energia.

Nesta etapa, o foco foi preparar o projeto para um fluxo de entrega mais próximo de um ambiente real, incluindo build automatizado, testes, Docker e workflows de deploy.

2. Arquitetura da aplicação

A API foi mantida separada em camadas para evitar que regras de negócio, endpoints e acesso ao banco fiquem misturados.



As principais entidades são Equipamento, HorarioOperacao, LeituraSensor, LogDesligamento e ConsumoEnergia.

3. Containerização

O Dockerfile utiliza multi-stage build. A primeira etapa usa o SDK .NET 8 para restaurar as dependências e publicar a aplicação. A imagem final utiliza apenas o runtime ASP.NET Core 8 e os arquivos publicados.

O ambiente local é orquestrado pelo docker-compose.yml e possui dois serviços:

- oracle-db: banco Oracle com volume persistente e healthcheck.
- ecocontrol-api: API ASP.NET Core publicada na porta 8080.

Os serviços compartilham a rede ecocontrol-network. A API aguarda o healthcheck do Oracle antes de iniciar, evitando tentativas de conexão enquanto o banco ainda está subindo.

As configurações sensíveis do ambiente local ficam no arquivo .env. O repositório mantém somente o .env.example.

4. Testes automatizados

O projeto de testes utiliza xUnit e Microsoft.AspNetCore.Mvc.Testing. Durante os testes, o provider Oracle é substituído pelo Entity Framework Core InMemory. Dessa forma, o pipeline consegue validar a API sem depender de um banco Oracle externo.

Entre os cenários previstos estão:

- consulta de equipamentos;
- consulta de logs de desligamento;
- consulta de consumo e resumo;
- consulta de leituras por equipamento;
- retorno 401 em operação administrativa sem autenticação;
- validação do endpoint /health.

5. Pipeline de integração contínua

O workflow ci.yml é executado em Pull Requests para develop e main e também em pushes para develop.

A execução segue esta ordem:

- checkout do repositório;
- configuração do .NET 8;
- restore das dependências;
- build da solução em modo Release;
- execução dos testes automatizados;
- upload dos resultados de teste;
- validação do build da imagem Docker.

Se build ou testes falharem, o job que valida a imagem Docker não avança.

6. Staging e produção

Os deploys foram separados em dois workflows para deixar claro quando uma versão está em homologação e quando foi promovida para produção.

Ambiente	**Branch**	**Tags da imagem**
Staging	develop	staging / staging-<sha>

Produção	main	latest / prod-<sha>

A tag contendo o SHA do commit serve para rastrear a versão exata do código que originou a imagem.

```
feature/*
  |
  v
Pull Request
  |
  v
develop ----> staging
  |
  v
Pull Request
  |
  v
main -------> produção
```

7. Configuração e segurança

As credenciais que estavam no arquivo de configuração foram externalizadas. No ambiente local são utilizadas variáveis do .env. Nos workflows, Docker Hub e Azure utilizam secrets configurados no repositório.

Principais variáveis previstas no pipeline:

```
DOCKERHUB_USERNAME
DOCKERHUB_TOKEN
AZURE_WEBAPP_NAME_STAGING
AZURE_WEBAPP_PUBLISH_PROFILE_STAGING
AZURE_WEBAPP_NAME_PRODUCTION
AZURE_WEBAPP_PUBLISH_PROFILE_PRODUCTION
```

8. Desafios encontrados

Durante a adaptação do projeto, alguns pontos precisaram ser ajustados:

- As credenciais estavam no appsettings.json e foram movidas para variáveis de ambiente e secrets.
- Os testes dependiam do Oracle e passaram a usar InMemory para funcionar de forma isolada no CI.
- A API poderia iniciar antes do banco; o Compose passou a usar healthcheck e depends_on.
- As imagens passaram a receber tags com o SHA do commit para facilitar a rastreabilidade.

9. Evidências de execução

Os prints devem ser obtidos a partir da execução real do projeto. Os espaços abaixo indicam quais evidências fazem parte da entrega.

1. Build e testes locais

Inserir evidência aqui.

2. Docker Compose e containers
Inserir evidência aqui.

3. Swagger e healthcheck local
Inserir evidência aqui.

4. Pipeline de CI
Inserir evidência aqui.

5. Deploy de staging
Inserir evidência aqui.

6. Staging em funcionamento
Inserir evidência aqui.

7. Deploy de produção
Inserir evidência aqui.

8. Produção em funcionamento
Inserir evidência aqui.

10. Checklist da entrega

Item	Status
Projeto compactado em ZIP com estrutura organizada	OK
Dockerfile incluído e pronto para build	OK
docker-compose.yml com aplicação, banco, volume, variáveis e rede	OK
Pipeline com build e testes	OK
Workflows de staging e produção	OK
README com instruções e descrição do pipeline	OK
Documentação técnica em PDF	OK
Evidências reais de staging e produção	Pendente

O último item deve ser marcado somente depois da execução dos dois ambientes e da inclusão dos respectivos prints.

11. Conclusão

A entrega mantém a proposta original do EcoControl e adiciona um fluxo de DevOps ao projeto. O código pode ser validado por testes automatizados, empacotado em uma imagem Docker e promovido entre staging e produção por workflows separados. A estrutura também mantém configurações sensíveis fora do código-fonte e facilita a identificação da versão implantada por meio do SHA do commit.